Save Point

Hosting the Web Build

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 27 October 2026

The app also builds for the web (`flutter build web`). The `Dockerfile` in the repo root builds it and serves the result as static files with nginx. The homelab setup is:

```
browser ──https──▶ Cloudflare Tunnel ──http──▶ Caddy :80 ──http──▶ alotanotes:80
                   (DNS + TLS for              (~/caddy, Docker     (this repo's
                    *.savepoint.co.nz)          Compose)             container)
```

Cloudflare owns DNS and TLS, and already routes the wildcard `*.savepoint.co.nz` through the tunnel to Caddy on port 80. So adding the app needs no DNS record and no certificate, only a Caddy site block.

Run it

Caddy and this container talk over a shared external Docker network called `web`, so no port is published on the host. Create the network once per host if it doesn't exist yet (Caddy's own compose file must also join it as an external network):

```
docker network create web
```

Then, from this repo on the homelab:

```
docker compose up -d --build
```

To deploy a new version, `git pull` and run the same command again.

The first build takes several minutes because it downloads the Flutter SDK (pinned to the same version as the CI release build, since there's no prebuilt image for it). Later builds reuse that layer.

Caddy

Add this to `~/caddy/Caddyfile`:

```
http://alotanotes.savepoint.co.nz {
    reverse_proxy alotanotes:80
}
```

Then reload Caddy:

```
cd ~/caddy && docker compose exec caddy caddy reload --config /etc/caddy/Caddyfile
```

(Adjust the service name and Caddyfile path if yours differ.)

The `http://` prefix matters. Without it, Caddy would try to get its own certificate for the domain, which fails behind the tunnel and isn't needed: Cloudflare already terminates TLS. `alotanotes` resolves through Docker's DNS on the `web` network, and `80` is the port nginx listens on inside the container.

Always use the HTTPS URL. The API key is stored with `flutter_secure_storage`, which on web uses the browser's WebCrypto API. Browsers only enable that on `https://` pages or `localhost`. Visitors through the tunnel get `https://`, so this is covered; it would only fail if you bypassed Cloudflare to reach the container directly.

How the web build differs

There's still no backend (see `future_work.md`). The server only hands out static files; **all data lives in each browser**:

- The database is SQLite compiled to WebAssembly (`sqflite_common_ffi_web`), stored in that browser's IndexedDB. Your phone and your PC each have their own separate notebook, and clearing site data for the domain erases it. Use Settings, then Export, to keep a copy.
- `web/sqlite3.wasm` and `web/sqflite_sw.js` are generated by `dart run sqflite_common_ffi_web:setup`. They're committed so a plain `flutter build web` works. Re-run the setup command after upgrading `sqflite_common_ffi_web`.
- AI requests go straight from the browser to the provider. OpenAI and Anthropic both allow that (Anthropic via the `anthropic-dangerous-direct-browser-access` header the app sends). A local Ollama/LM Studio endpoint has to allow the site's origin itself, e.g. `OLLAMA_ORIGINS=https://alotanotes.savepoint.co.nz`.
- The `admin` / `1234` login is still only a local placeholder. It doesn't protect anything on the server (there's nothing on the server to protect). Anyone who can reach the URL gets their own empty copy of the app, and each person uses their own API key.